

Aula 1 - Self Attention in Transformer Neural Networks

RNNs

RNNs (Recurrent Neural Networks) costumavam ser o estado da arte para modelagem sequência-para-sequência. Muitas aplicações em processamento de linguagem natural.

Mas sofrem de 2 problemas.

1) São muito lentas. 2) O token t pode olhar apenas para o contexto $t-1$ ou $t+1$ separadamente, e depois concatenar. Isso tira qualidade da informação. Resolve-se isso com o mecanismo Attention.

Attention Mechanism

Com Attention, podemos decidir em quais partes cada palavra precisa focar.

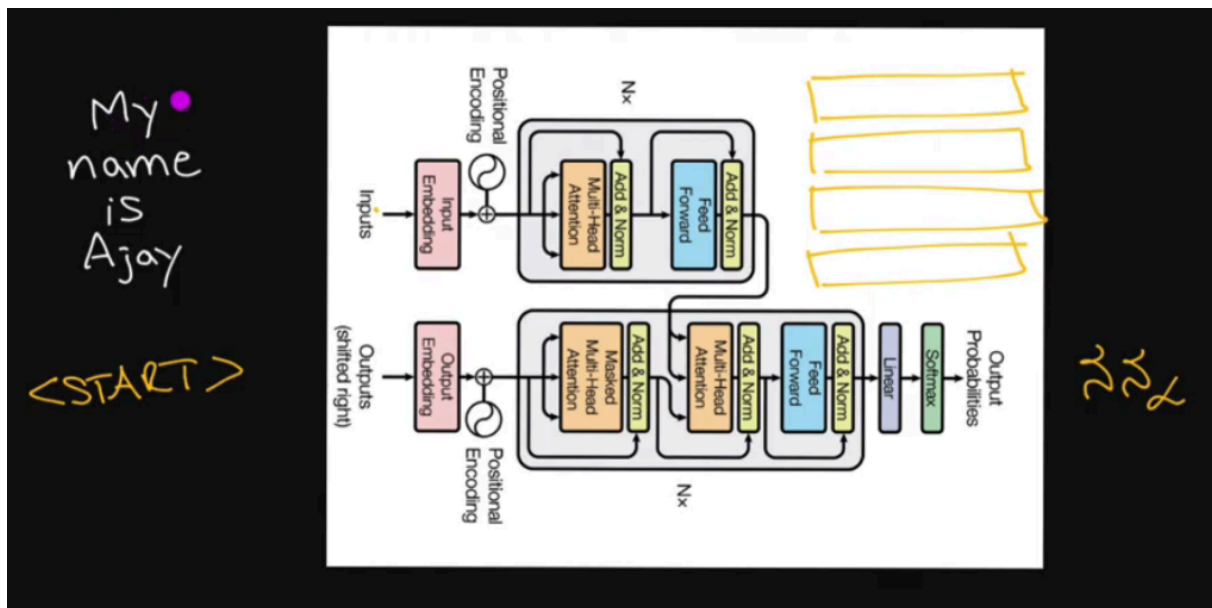


Quanto mais claro, mais atenção. (maior o número)

- My - Foca em “My” e “Name”
- “Name” - Foca muito em “Ajay”

Transformer Overview

Vamos traduzir “My name is Ajay”.



1. Passamos todas as palavras pelo Encoder.
2. Isso vai gerar 4 vetores (um para cada palavra)
3. Na entrada do Decoder, vai o token <start>
4. No output, teremos “Meu”
5. Na entrada do Decoder, vai o token “Meu”
6. No output, teremos “nome”
7. Na entrada do Decoder, vai o token “nome”
8. No output, teremos “é”
9. Na entrada do Decoder, teremos “é”
10. No output, teremos “Ajay”

Encoder - Token Embedding + Positional Encoding

Entrada: “My Name is Ajay”

1. **Token Embedding** = Significado Semântico da palavra (palavra vira vetor)

$$e_{\text{Name}} = [0.25, -0.80, 0.15, 0.90]$$

2. **Position Encoding** = Vetor de mesmo tamanho representando a posição do token na frase

$$p_1 = [0.00, 1.00, 0.50, -0.50]$$

Vetor de Entrada = Embedding do Token + Positional Encoding

$$\text{Vetor Final} = [0.25, 0.20, 0.65, 0.40]$$

No paper, cada vetor de entrada tem dimensão 512.

Encoder - Attention

- Q - O que estou procurando (d_k)
- K - O que posso oferecer (d_k)
- V - O que realmente ofereço (d_v)

$$\text{Self Attention} = \text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M \right) V$$

- $\sqrt{d_k}$ minimiza a variância e estabiliza os valores.

Aula 2 - Multi-Head Attention

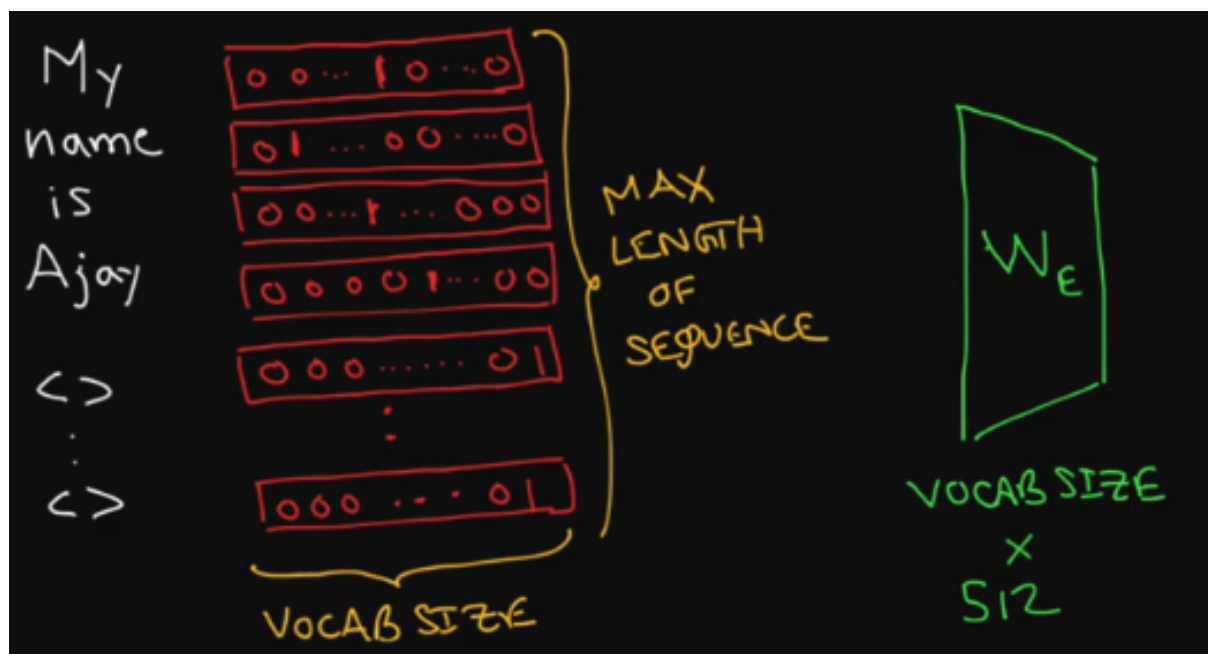
$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_2)W^O$$

1. Token Embedding + Positional Encoding gera um vetor $d_{\text{model}} = 512$.
2. Geramos 3 matrizes Q,K,V a partir desse vetor, com $d_k = d_v = 512$
3. Para Q,K,V, quebramos em 8 peças de $d_i = 64$. Cada uma vai para uma unidade de Self-Attention e vai gerar um vetor final.
4. Os vetores são concatenados (unidos).
5. Uma camada linear final transforma a concatenação bruta em uma representação final refinada (mistura o que foi concatenado). O resultado é um vetor com um contexto muito mais rico.

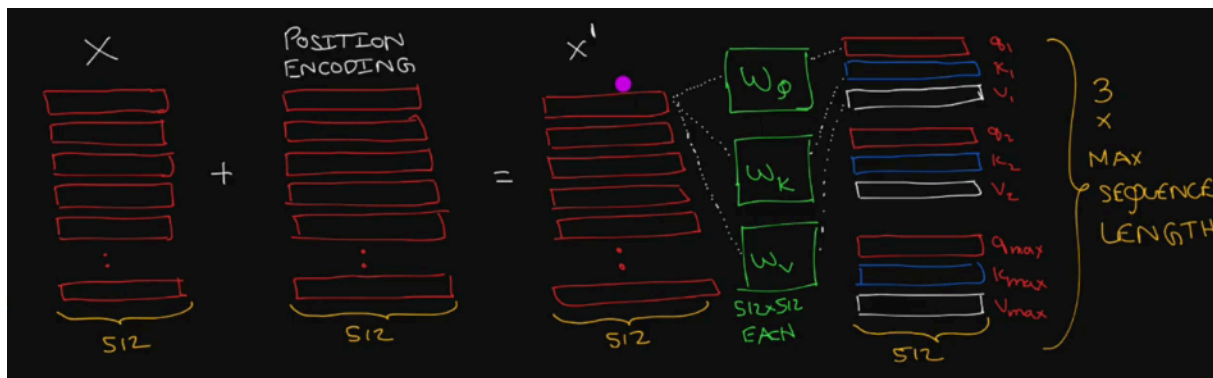
Aula 3 - Positional Encoding

Transformer Architecture

Primeiro, temos a arquitetura do transformer:



1. Cada palavra/token é transformada em um vetor de tamanho 512, o tamanho do vocabulário.
2. Depois, adiciona-se a cada vetor soma-se um position encoding de mesmo tamanho.
3. Com esse vetor final, cada palavra w será transformada em 3 vetores w_q, w_k, w_v (Query, Key, Value), através de uma projeção linear. Quando juntamos tudo, temos as matrizes Q, K, V.



Nota: Projeções Lineares (Q, K, V)

Para obter as matrizes Q, K, V , uma rede neural linear foi treinada, de modo que, para uma matriz de tokens $x \in \mathbb{R}^{n \times d_{\text{model}}}$:

$$Q = XW_Q + b_Q$$

$$K = XW_K + b_K$$

$$V = XW_V + b_V$$

Os pesos W_Q, W_K, W_V são os parâmetros treinados junto com o resto da rede via backpropagation.

Positional Embedding

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$\text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

- i = Componente do vetor de embedding (0-512)
- d_{model} = Embedding Length
- pos = Posição do token na sequência (0,1,2,3,...)

Por que essas fórmulas?

1. Periodicidade
2. Valores limitados (entre -1 e 1)
3. Fácil de extrapolar para sequências longas

Em uma matriz onde cada linha é um embedding:

- O pos varia na vertical.
- O i varia na horizontal e altera a frequência da onda (frequências altas nas primeiras dimensões, frequências baixas nas últimas)

Sem o positional embedding, as frases:

- “O cão comeu o gato”
- “O gato comeu o cão”

Produziriam o mesmo resultado, pois contém ambas as mesmas palavras. Com Positional Embedding, os vetores iniciais de cada frase serão diferentes e, portanto, serão tratados de maneiras diferentes.

Aula 4 - Layer Normalization

Transformer Architecture

1. Cada palavra/token é transformada em um vetor de tamanho 512, o tamanho do vocabulário.
2. Depois, adiciona-se a cada vetor soma-se um position encoding de mesmo tamanho.
3. Com esse vetor final, cada palavra w será transformada em 3 vetores w_q, w_k, w_v (Query, Key, Value), através de uma projeção linear. Quando juntamos tudo, temos as matrizes Q, K, V.
4. Todos os vetores w_q, w_k, w_v são divididos em 8 partes ($\frac{512}{8} = 64$). Cada parte será um vetor para um bloco de atenção (são 8 blocos)
5. Os vetores serão concatenados, em ordem. A saída terá tamanho 512 novamente.
6. Esse vetor enriquecido será somado com o vetor vocabulário original (com PE), e normalizado. Essa soma é para evitar o vanishing gradiente (gradientes de valores pequenos tendem a zero, parando o aprendizado)

Layer Normalization

LayerNorm[$x' + \text{out}$]

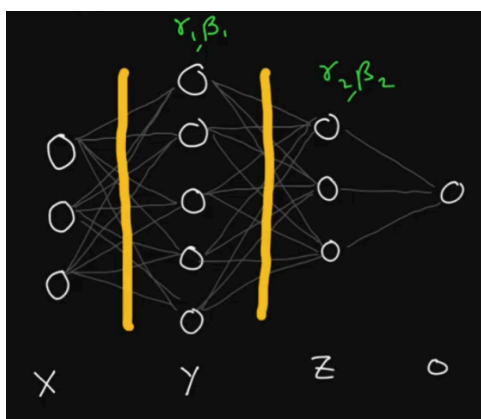
(Add and Norm)

- x' = Vetor Original + Positional Encoding
- out = Saída concatenada dos blocos de atenção

Layer Normalization: What?

A normalização encapsula os valores em um intervalo bem menor do que o original, com centro tipicamente em torno de zero. Isso permite um treino muito mais estável, com passos mais uniformes durante o backpropagation.

Suponha uma rede neural.



$$x' = f(W_1^T x + b_1)$$

A normalização para a próxima camada será:

$$y = \gamma_1 \left[\frac{x' - \mu_1}{\sigma_1} \right] + \beta_1$$

- μ_1 = Média dos valores de ativação da camada
- σ_1 = Standard Deviation dos valores de ativação da camada
- γ_1 (escala), β_1 (deslocamento) = Parâmetros aprendíveis do próprio LayerNorm. O mesmo para toda a camada.

A normalização pura força os dados a terem estritamente média 0 e variância 1. No entanto, essa distribuição fixa nem sempre é a ideal para a camada seguinte aprender.

Os parâmetros γ e β dão à rede a liberdade de desfazer ou ajustar a normalização se necessário:

- Se a rede aprender que $\gamma = 1$ e $\beta = 0$, ela mantém a normalização perfeita ($\mu = 0, \sigma^2 = 1$).
- Se a rede aprender que $\gamma = \sigma_{\text{original}}$ e $\beta = \mu_{\text{original}}$, ela pode restaurar os dados exatamente ao seu estado original.

Layer Normalization: How?

$$x' = \begin{pmatrix} 0.2 & 0.1 & 0.3 \\ 0.5 & 0.1 & 0.1 \end{pmatrix}$$

$$\mu_{11} = \frac{1}{3}[0.2 + 0.1 + 0.3] = 0.2$$

$$\mu_{21} = \frac{1}{3}[0.5 + 0.1 + 0.1] = 0.233$$

$$\sigma_{11} = \sqrt{\frac{1}{3}\{[0.2 - 0.2]^2 + [0.1 - 0.2]^2 + [0.3 - 0.2]^2\}} = 0.08164$$

$$\sigma_{21} = \sqrt{\frac{1}{3}\{[0.5 - 0.233]^2 + [0.1 - 0.233]^2 + [0.1 - 0.233]^2\}} = 0.1885$$

$$\mu = \begin{pmatrix} \mu_{11} \\ \mu_{21} \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.233 \end{pmatrix}$$

$$\sigma = (\sigma_{11} + \sigma(21)) = \begin{pmatrix} 0.08164 \\ 0.1885 \end{pmatrix}$$

$$y = \frac{x' - \mu}{\sigma} = \begin{pmatrix} 0 & -1.2248 & 1.2248 \\ 1.414 & -0.707 & -0.707 \end{pmatrix}$$

$$\text{out} = \gamma \cdot y + \beta$$

Aula 5 - Blowing up the Transformer Encoder! (ENCODER COMPLETO)

Obs: Linhas = max_sequence_lenght, Colunas = d_k

ENTRADA DO ENCODER

1. Input Sentence

TRATAMENTO DE INPUT

2. **Embedding Transform** -> Transformar palavras/tokens em vetores de dimensão $d = 512$ (vocabulário = 2^{512} possibilidades de representação)
3. **Positional Encoding** -> Adiciona-se ao embedding valores periódicos que informam a ordem/posição das palavras na sequência

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

ENCODER

1. **Generate Q,K,V** -> Cada vetor sofre uma **projeção linear** para gerar os vetores w_q, w_k, w_v .
Juntando todos os vetores em uma matriz:

$$Q = XW_Q + b_Q$$

$$K = XW_K + b_K$$

$$V = XW_V + b_V$$

(onde W_Q, W_K, W_V são matrizes aprendíveis que serão ajustadas pelo backpropagation)

2. **Multi-Head** -> Cada vetor será quebrado em 8 blocos ($d = 64$), para processamento paralelo e mais enriquecido em contexto.
3. **Multi-Head Self Attention** -> Aplica Attention a cada uma das Heads. (A 1° linha de Q vezes a 2° linha de K tranposta, etc)

$$\text{Self Attention} = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M\right)V$$

4. **Concatenate** -> Os 8 blocos são concatenados novamente

$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{Self-Att}_1, \dots, \text{Self-Att}_n)W^O$$

5. **Linear-Layer** -> W^O mistura as informações concatenadas, enriquecendo o vetor final.
6. **Add** -> Resultado da Concatenação + (Input Embedding + Positional Encoding). Soma-se o Attention ao Input inicial, evitando Vanishing Gradient (valores pequenos demais).
7. **Layer Normalization** -> Estabiliza os valores, os deixando em torno de zero. Para cada $\text{LayerNorm}()$, existe um único tensor γ e β .

$$y = \gamma \left[\frac{x - \mu}{\sigma} \right] + \beta$$

(onde γ, β são parâmetros aprendíveis)

8. **Linear + ReLU + Dropout** -> Linear é uma rede neural de camada única. Dropout é desligar neurônios aleatoriamente, zerando uma porcentagem das saídas deles, o que evita vício em padrões e permite generalizar. Entrada: $\text{max_seq} \times 512$. Saída: $\text{max_seq} \times 1024$

$$y = \text{ReLU}(W_1x + b_1)$$

9. **Linear** -> Outra rede neural de camada única, para comprimir de volta para 512. Entrada: $\text{max_seq} \times 1024$. Saída: $\text{max_seq} \times 512$.

$$y = W_2x + b_2$$

10. **Add** -> Resultado + Input da Rede neural
11. **Layer Normalization** -> Estabiliza os valores.

$$y = \gamma \left[\frac{x - \mu}{\sigma} \right] + \beta$$

12. Os vetores finais são uma representação muito mais rica em contexto do que os vetores iniciais.

13. Fazemos tudo de novo (por ex, 12 vezes), para enriquecer ainda mais os vetores.
14. Output: Vetor $\text{max_sequence_length} \times 512$ ($\text{max_sequence_length} = n$ Tokens)
15. Repetição

ENTRADA DO DECODER

1. Input -> A entrada é o que já foi gerado.
 - Padding serve para fixar tamanho da entrada
 - Tamanho: $\text{batch_size} \times \text{max_seq_length} \times d_{\text{model}}$

<start> first second third <end> <padding> <padding> ... <padding>

2. **Positional Encoding** -> Adiciona-se ao embedding valores periódicos que informam a ordem/ posição das palavras na sequência

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$\text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

DECODER

3. **Generate Q,K,V** -> Cada vetor sofre uma **projeção linear** para gerar os vetores w_q, w_k, w_v .
Juntando todos os vetores em uma matriz:

$$Q = XW_Q + b_Q$$

$$K = XW_K + b_K$$

$$V = XW_V + b_V$$

4. **Multi-Head** -> Cada vetor será quebrado em 8 blocos ($d = 64$), para processamento paralelo e mais enriquecido em contexto.
5. **Masked Self Attention** -> Aplica-se máscara, uma matriz triangular superior de $-\infty$'s, de modo que evita "trapaça" durante a tentativa de inferência, ocultando a palavra a ser adivinhada. usa-se $-\infty$ por causa da softmax.
6. **Multi-Head Self Attention** -> Aplica Attention a cada uma das Heads. (A 1° linha de Q vezes a 2° linha de K tranposta, etc)

$$\text{Self Attention} = \text{Softmax}\left(\frac{Q \cdot K^T + M}{\sqrt{d_k}} + M\right)V$$

7. **Concatenate** -> Os 8 blocos são concatenados novamente

$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{Self-Att}_1, \dots, \text{Self-Att}_n)W^O$$

8. **Linear-Layer** -> W^O mistura as informações concatedadas, enriquecendo o vetor final.
9. **Add** -> Resultado da Concatenação + (Input Embedding + Positional Encoding). Soma-se o Attention ao Input inicial, evitando Vanishing Gradient (valores pequenos demais).
10. **Layer Normalization** -> Estabiliza os valores, os deixando em torno de zero. Para cada $\text{LayerNorm}()$, existe um único tensor γ e β .

$$y = \gamma\left[\frac{x - \mu}{\sigma}\right] + \beta$$

11. **Generate Q,K,V** -> Cada vetor sofre uma **projeção linear** para gerar os vetores w_q, w_k, w_v . Q é gerado a partir do output anterior, e K/V a partir do Encoder. Juntando todos os vetores em uma matriz:

$$Q = X_D W_Q + b_Q$$

$$K = X_E W_K + b_K$$

$$V = X_E W_V + b_V$$

12. **Multi-Head** -> Cada vetor será quebrado em 8 blocos ($d = 64$), para processamento paralelo e mais enriquecido em contexto.
13. **Multi-Head Cross Attention** -> Aplica Attention a cada uma das Heads. (A 1° linha de Q vezes a 2° linha de K tranposta, etc). Aqui, não precisa-se de máscara (Encoder funciona como um mapa)

$$\text{Self Attention} = \text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M \right) V$$

14. **Concatenate** -> Os 8 blocos são concatenados novamente
15. **Linear-Layer** -> W^O mistura as informações concatenatedas, enriquecendo o vetor final.

$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{Self-Att}_1, \dots, \text{Self-Att}_n) W^O$$

16. **Add** -> Resultado da Concatenação + Output anterior. Evita Vanishing Gradient (valores pequenos demais).
17. **Layer Normalization** -> Estabiliza os valores, os deixando em torno de zero. Para cada $\text{LayerNorm}()$, existe um único tensor γ e β .

$$y = \gamma \left[\frac{x - \mu}{\sigma} \right] + \beta$$

18. **Linear + ReLU + Dropout** -> Linear é uma rede neural de camada única. Dropout é desligar neurônios aleatoriamente, zerando uma porcentagem das saídas deles, o que evita vício em padrões e permite generalizar. Entrada: $\text{max_seq} \times 512$. Saída: $\text{max_seq} \times 1024$

$$y = \text{ReLU}(W_1 x + b_1)$$

19. **Linear** -> Outra rede neural de camada única, para comprimir de volta para 512. Entrada: $\text{max_seq} \times 1024$. Saída: $\text{max_seq} \times 512$.

$$y = W_2 x + b_2$$

20. **Add** -> Resultado + Input da Rede neural
21. **Layer Normalization** -> Estabiliza os valores.
22. Repetição

$$y = \gamma \left[\frac{x - \mu}{\sigma} \right] + \beta$$

SAÍDA DO DECODER

1. **Linear** -> Rede neural de camada única, para transformar 512 (d_{model}) em $\text{translation_vocab_size}$. Isso mapeia cada palavra para o vocabulário da linguagem final.
2. **Softmax** -> Saída de probabilidades da próxima palavra a ser gerada.

Aula 6: Sentence Tokenization in Transformer Code

Nota Minha: Transformers é apenas sobre backpropagation.

O que faz o Transformer aprender é o **backpropagation**. Essa é a única coisa certa disso tudo: **gradiente descendente em direção a mínimos locais para minimizar a função custo**.

O resto são técnicas para tentar melhorar esse aprendizado, “conceitos” que queremos que a rede tente aprender.

- O positional encoding faz permutações de uma mesma frase terem números diferentes.
- As matrizes Q, K, V não são nada no começo. Mas a lógica é uma $\text{softmax}(Q, K)V$ obriga Q e K a se comportarem como “query-key” e V a se comportar como algo a ser enriquecido por isso de “query-key”. Isso de “prestar atenção” é mais algo intuitivo do que certo.
- Multi-Head é dividir em blocos menores para um processamento mais localizado, melhorando a qualidade da análise
- Add + LayerNorm é uma técnica comum para facilitar o aprendizado (melhora efeito do gradiente)
- No LayerNorm, γ, β são otimizados para cumprir com a função custo. Dá-se a eles sentido de “escalar” e “deslocamento”, mas é só uma intuição. Apenas estamos dando mais possibilidades de aprendizado pro backpropagation.

Isso se chama **Viés Indutivo** (Inductive Bias), suposições/facilidades que embutimos na arquitetura para forçar ou guiar a rede a aprender determinados padrões em vez de tentar aprender tudo do zero absoluto. Todas essas camadas, somas residuais, encodings e normalizações são o **nosso jeito de encurtar o caminho do gradiente** e viabilizar a otimização em tempo útil.

Dizemos à rede: “A estrutura do seu cálculo será matrizes Q, K, V passando por um softmax”. O Backpropagation encontra o conteúdo: O otimizador preenche esse molde com os valores exatos de parâmetros que resolvem o problema.

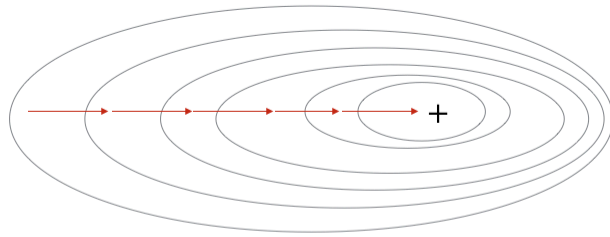
Transformer Encoder in 100 lines of code!

Batch Data

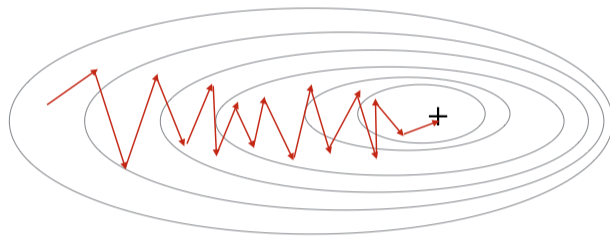
No treinamento, passamos múltiplos inputs (dataset) ao mesmo tempo. Esses múltiplos inputs constituem um batch (lote).

- **Batch Gradient Descent (BGD)** - Usa todo o conjunto de dados. 1 atualização por época (calcula o erro de todos os exemplos, calcula a média dos gradientes e atualiza). Caminho direto e estável até o mínimo, mas inviável para datasets gigantes.
- **Stochastic Gradient Descent (SGD)** - 1 única amostra aleatória por vez. N atualizações por época, de N amostras (para cada exemplo, calcula o gradiente e atualiza). Caminho rápido e caótico, computacionalmente leve mas com oscilação em torno do ponto ótimo.
- **Mini-batch** - Pequeno lote de $n < N$ amostras aleatórias por vez. N/n atualizações por época. É o meio termo.

Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent

